Python 28 天 • 基础期

基础期 • 第 1-9 天

Python 语法、函数、文件 IO、numpy、pandas 入门

共 9 天：第1天、第2天、第3天、第4天、第5天、第6天、第7天、第8天、第9天

每一份前面的深色页 = 新的一天开始；后面几页就是当天那份知识点 PPT 的原文。

配套：① 开始今天的学习.cmd 每天会自动打开当天那一份；这本合订本是给你周末连着翻、面试前一晚整个过一遍用的。

这本里都有哪几天

第1天 脚本骨架：变量、四种类型、格式化、输入
11/2

第2天 分支与循环：ifelif、whilefor、range、breakcontinue
11/3

第3天 容器四件套：list tuple dict set，切片与排序
11/4

第4天 ★Excel 一天速成：数据透视表 + XLOOKUPVLOOKUP + SUMIFS
11/5

第5天 函数：参数、返回值、作用域、lambda
11/6

第6天 面向对象 + 模块与包 + 常用标准库
11/7

第7天 文件 IO + 异常处理 + CSVJSON 读写（W4 收尾验收）
11/8

第8天 numpy：ndarray、向量化、广播、布尔索引
11/9

第9天 pandas 读数据 + SeriesDataFrame + 一眼看清一张表
11/10

基础期 · 第 1 天 · 11/2

脚本骨架：变量、四种类型、格式化、输入

写一个 .py 文件，收两个输入、算出一个结果、用 f-string 格式化输出，全程不查语法。

第 1 天 · 11/2 · 今日知识点回顾

脚本骨架：变量、四种类型、格式化、输入

目标：写一个 .py 文件，收两个输入、算出一个结果、用 f-string 格式化输出，全程不查语法。

对着 B 站视频看：视频讲到的每一句，这张 PPT 里都有一条；复习时只翻这份，不用再回去翻分P。

今天结束时，你应该能做到

验收标准（做到这句就算过关）

- 写一个 .py 文件，收两个输入、算出一个结果、用 f-string 格式化输出，全程不查语法。

今天的做法（顺序别颠倒）

- 本机 VSCode 里先写这几段（热手 + 验证概念）：
- 题库地址：file:///D:/DSH工作区/00-学习中心/60-练习环境/Python-Env/data
- （今天的练习数据都在这个目录；明天起真正开始读它）
- 做完记一句：今天卡在哪、错在哪 → 写进 3-每日打卡。

今天看哪几集、读哪一章

这一天的知识：5 集 · 合计约 55 分钟（边看边在本机敲一遍）

P14	字面量	约 9 分钟
P16	变量	约 12 分钟
P17	数据类型	约 10 分钟
P23	字符串格式化	约 13 分钟
P28	数据输入(input语句)	约 11 分钟

教材（已下载到 20-教材与讲义\python-data\，断网也能看） https://raw.githubusercontent.com/jackfrued/Python-100-Days/master/Day01-20/03.Python语言中的变量.md

视频地址就在下面的「今天做哪些题」网页里，点一下直达分P。

知识点 1/7 • 变量没有类型，值才有

- 一句话：Python 的变量只是「贴在对象上的名字」，换个值就换个类型，赋值时不报错。
- 代码：

```
x = 100
print(type(x), x)      # <class 'int'> 100
x = '100'
print(type(x), x)      # <class 'str'> 100   ← 同一个名字，换了类型，不报错
```

★ 这是和 SQL 最大的差别：SQL 的列类型固定，塞错类型当场报错；Python 把类型错误推迟到「你真正用它算的时候」才炸 —— 所以「跑通了」不等于「对了」。

知识点 2/7 • 四种基本类型，以及怎么转

- 一句话：int（整数）/ float（小数）/ str（文本）/ bool（真假日），转类型用同名函数包一下。
- 代码：

```
print(int('12') + 1)          # 13
print(float('12.5') * 2)      # 25.0
print(str(12) + '5')          # '125'  ← 字符串相加是拼接，不是 17
print(int(12.9))              # 12     ← 直接砍掉小数，不四舍五入
```

★ int('12.5') 会报 ValueError: invalid literal for int() with base 10: '12.5'—— 带小数点的字符串不能直接转 int，要先 float 再 int。这条坑清洗数据时天天踩。

知识点 3/7 • 三种除法，别混

- 一句话：/ 是真除法（得小数），// 是整除（向下取整），% 是取余。
- 代码：

```
print(7 / 2)     # 3.5
print(7 // 2)    # 3
print(7 % 2)     # 1
print(-7 // 2)   # -4   ← 不是 -3，// 是「向下」取整，不是「向零」
```

★ 算「第几页」「第几组」时用 //，算「能不能整除」时用 %；两个都用错会差 1。

知识点 4/7 • 字符串三种写法，f-string 是主力

- 一句话：单引号、双引号等价；三引号能跨行；格式化一律用 f'{变量}'。
- 代码：
- name, age, price = '张三', 23, 1899.0

```
print(f'{name} 今年 {age} 岁')
print(f'{price:.2f} 元，打九折 {price * 0.9:.2f} 元')   # 1899.00 元，打九折 1709.10 元
print(f'{1234567:,}')                                    # 1,234,567
```

★ f-string 的冒号后面是「格式说明」：.2f 两位小数、，千分位、>10 右对齐补空格。

- 写报表时这三个够用了，别再回去用 % 和 .format。

知识点 5/7 • input 拿到的永远是字符串

- 一句话：input() 返回 str，要算数必须先转，转不了就报 ValueError。
- 代码：

```
h = float(input('身高(m): '))     # 用户敲 1.75 → 得到 1.75
w = float(input('体重(kg): '))
print(f'BMI = {w / h ** 2:.2f}')
```

★ 直接 int(input()) 而用户敲了空回车 → ValueError: invalid literal for int() with base 10: '' 真实脚本里要用 try/except 兜住（第 7 天专门讲）。

知识点 6/7 • 变量命名与注释

- 一句话：小写字母+下划线；不能数字开头；不能是关键字；注释用 #。
- 代码：

```
order_total = 0          # 好
# 2order = 0             # 语法错：不能数字开头
# class = 'A'            # 语法错：class 是关键字
```

★ 中文变量名 Python 3 允许（订单总额 = 100 能跑），但报错信息里中文会变成一串转义，排查时更费劲 —— 一律用英文。

知识点 7/7 • 第一个「真数据」：用最笨的办法读 CSV

- 一句话：CSV 就是逗号分隔的文本，一行一条记录，第一行是表头。
- 代码：

```
import pathlib
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
with open(DATA / 'orders_dirty.csv', encoding='utf-8-sig') as f:
for i, line in enumerate(f):
print(line.rstrip().split(',')[:6])
if i >= 3:
break
```

- （encoding='utf-8-sig' 是为了吃掉 Excel 存 CSV 时写在开头的 BOM，不写的话第一个列名会变成 '\ufefforder_id'，后面按列名取值全部 KeyError。）

代码速查：今天这几段，要在本机跑通（1/4）

```
import pathlib
```

```
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
```

1) 变量换类型

x = 100

```
print(type(x), x)
```

x = '100'

代码速查：今天这几段，要在本机跑通（2/4）

```
print(type(x), x, int(x) + 1)
```

```
# 2) 格式化：把 1899.0 输出成「1,899.00 元，打九折 1,709.10 元」
```

price = 1899.0

```
print(f'{price:,.2f} 元，打九折 {price * 0.9:,.2f} 元')
```

3) 输入 + 计算（跑的时候手动敲 1.75 和 70）

h = float(input('身高(m)：'))

w = float(input('体重(kg)：'))

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（3/4）

```
print(f'BMI = {w / h ** 2:.2f}')
```

4) 用最笨的办法读真数据的前 4 行

```
with open(DATA / 'orders_dirty.csv', encoding='utf-8-sig') as f:
```

for i, line in enumerate(f):

```
        print(line.rstrip().split(',')[:6])
```

if i >= 3:

break

代码速查：今天这几段，要在本机跑通（4/4）

验收（做到才算过关）：

① 上面 4 段全部跑出结果，截图或复制输出留一份；

② 能不看资料说出 int('12.5') 为什么会报错、怎么绕过；

③ 能把 -7 // 2 的结果和原因讲清楚。

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

最容易错的地方

★ 这是和 SQL 最大的差别：SQL 的列类型固定，塞错类型当场报错；Python 把类型错误推迟到「你真正用它算的时候」才炸 —— 所以「跑通了」不等于「对了」。

★ 算「第几页」「第几组」时用 //，算「能不能整除」时用 %；两个都用错会差 1。

写报表时这三个够用了，别再回去用 % 和 .format。

★ 中文变量名 Python 3 允许（订单总额 = 100 能跑），但报错信息里中文会变成一串转义，排查时更费劲 —— 一律用英文。

这些句子都是「差一个字、结果差一截」的地方，考前只翻这一页也值。

合上视频，自测一遍（能打勾才算学会）

- [ ] 我能不看资料写出：算「第几页」「第几组」时用 //，算「能不能整除」时用 %；两个都用错会差 1。
- [ ] 我能不看资料写出：中文变量名 Python 3 允许（订单总额 = 100 能跑），但报错信息里中文会变成一串转义，排查时更费劲 —— 一律用英文。
- [ ] 我能不看资料说出：写报表时这三个够用了，别再回去用 % 和 .format。
- [ ] 我能不看资料说出：一句话：input() 返回 str，要算数必须先转，转不了就报 ValueError。
- [ ] 我能不看题源，把今天用到的每个函数「为什么要有它」说一遍
- [ ] 我能说出今天每个参数/返回值改了会怎样——换个写法结果差在哪

这些句子都是「差一个字、结果差一截」的地方；三个勾以上，今天就算过关。

今天收工前，先知道明天学什么

第 2 天 • 分支与循环：if/elif、while/for、range、break/continue

不用提前学。明天点「① 开始今天的学习」，会换上明天那份 PPT。

收工三件事（5 分钟）

- 今天没跑通的脚本存进 90-我的作业\，命名 2026-11-02-第01天-*.py
- 打卡表「③ 每日打卡」记一笔：今天卡在哪、错在哪（一句话就够）
- 错题明天重做一遍，练习数据在 60-练习环境\Python-Env\data

基础期 · 第 2 天 · 11/3

分支与循环：ifelif、whilefor、range、breakcontinue

不看资料写出「猜数字」和「九九乘法表」两个小程序，并说清 for 和 while 各该在什么场合用。

第 2 天 · 11/3 · 今日知识点回顾

分支与循环：if/elif、while/for、range、break/continue

目标：不看资料写出「猜数字」和「九九乘法表」两个小程序，并说清 for 和 while 各该在什么场合用。

对着 B 站视频看：视频讲到的每一句，这张 PPT 里都有一条；复习时只翻这份，不用再回去翻分P。

今天结束时，你应该能做到

验收标准（做到这句就算过关）

- 不看资料写出「猜数字」和「九九乘法表」两个小程序，并说清 for 和 while 各该在什么场合用。

今天的做法（顺序别颠倒）

- 本机 VSCode 里写这三段，每段都要跑出结果：
- 题库地址：file:///D:/DSH工作区/00-学习中心/60-练习环境/Python-Env/data
- （明天开始用这份数据做分支/循环的过滤统计）
- 做完记一句：今天卡在哪、错在哪 → 写进 3-每日打卡。

今天看哪几集、读哪一章

这一天的知识：6 集 · 合计约 79 分钟（边看边在本机敲一遍）

P29	布尔类型和比较运算符	约 18 分钟
P30	if语句的基本格式	约 10 分钟
P34	if_elif_else组合使用的语法	约 17 分钟
P43	for循环的基础语法	约 10 分钟
P45	range语句	约 12 分钟
P49	continue和break	约 12 分钟

教材（已下载到 20-教材与讲义\python-data\，断网也能看） https://raw.githubusercontent.com/jackfrued/Python-100-Days/master/Day01-20/05.分支结构.md

视频地址就在下面的「今天做哪些题」网页里，点一下直达分P。

知识点 1/8 • Python 靠缩进分块，不靠大括号

- 一句话：冒号后面缩进 4 个空格就是「属于这个 if / for 的代码」，缩进错了逻辑就错了。
- 代码：

```
score = 72
if score >= 60:
print('及格')
print('这句也在 if 里')
print('这句一定执行')
```

★ 混用 Tab 和空格会报 TabError: inconsistent use of tabs and spaces in indentation。

- VSCode 右下角把缩进设成「空格: 4」，一辈子不用再管这条。

知识点 2/8 • 比较与逻辑运算：and 比 or 先算

- 一句话：and 优先级高于 or，混用必须加括号。
- 代码：
- age, city = 28, '北京'

```
print(age > 25 and city == '北京')          # True
print(city == '上海' or age > 25 and age < 30)   # True — 先算 and
print((city == '上海' or age > 25) and age < 30) # True — 括号改顺序
```

★ 和 SQL 的 AND/OR 优先级规则一模一样 —— 这条你已经会了，只是换个写法。

知识点 3/8 • if / elif / else：从上往下，命中即停

- 一句话：elif 是「否则如果」，只会执行第一个成立的分支，后面的不再判断。
- 代码：

```
score = 72
if score >= 90:
grade = 'A'
elif score >= 75:
grade = 'B'
elif score >= 60:
grade = 'C'
else:
grade = 'D'
print(grade)     # C
```

知识点 3/8 • if / elif / else：从上往下，命中即停（续）

★ 顺序写反就全错：如果把 >= 60 放在最前面，100 分也会被判成 C。

- 分档判断永远「从严到宽」排。

知识点 4/8 • for + range：已知次数用它

- 一句话：for i in range(n) 循环 n 次，i 从 0 到 n-1；range(1, 11) 是 1 到 10。
- 代码：

```
for i in range(3):
print(i)              # 0 1 2
for i in range(1, 11):
print(i, end=' ')     # 1 2 3 4 5 6 7 8 9 10
print()
print(sum(range(1, 101))) # 5050
```

★ range 是「左闭右开」：range(1, 11) 到 10 为止。SQL 的 BETWEEN 是闭区间，两边的「含不含尾巴」正好相反，切换语言时最容易差 1。

知识点 5/8 • while：不知道次数、靠条件停

- 一句话：while 后面跟条件，条件为真就一直转；一定要有能让条件变假的语句。
- 代码：
- n, total = 1, 0

```
while n <= 100:
```

知识点 5/8 • while：不知道次数、靠条件停（续）

- total += nn += 1

```
print(total)      # 5050
```

★ 忘了 n += 1 就是死循环 —— 按 Ctrl+C 停。VSCode 里用「运行和调试」面板跑，比在终端里跑更好中断。

知识点 6/8 • break 与 continue：一个跳出、一个跳过

- 一句话：break 结束整个循环；continue 只跳过本轮，接着下一轮。
- 代码：

```
for i in range(1, 10):
if i == 3:
continue        # 3 被跳过
if i == 7:
break           # 到 7 就整个结束
print(i, end=' ')   # 1 2 4 5 6
print()
```

★ for-else 这个语法（循环没被 break 打断才执行 else）Python 有，但可读性差 ——面试写代码时别用，评委要反应两秒。

知识点 7/8 • 列表推导式：一行代替「建空表 + 循环 + append」

- 一句话：[表达式 for 变量 in 可迭代对象 if 条件]，这是 pandas 之前最重要的写法。
- 代码：

```
squares = [i * i for i in range(5)]
print(squares)                          # [0, 1, 4, 9, 16]
evens = [i for i in range(20) if i % 2 == 0]
print(evens)                            # [0, 2, 4, ..., 18]
names = ['张三', '李四', '王五']
print([n + '同学' for n in names])     # ['张三同学', '李四同学', '王五同学']
```

★ 推导式只能放表达式，不能放 print、不能放赋值。要写两句以上就用普通 for 循环，硬塞进推导式只会让代码更难读（也更容易写错）。

知识点 8/8 • enumerate 与 zip：两个最常用的搭档

- 一句话：enumerate 同时给下标和值；zip 把两个列表按位置配对。
- 代码：

```
cities = ['北京', '上海', '广州']
for i, c in enumerate(cities, start=1):
print(i, c)                    # 1 北京 / 2 上海 / 3 广州
amounts = [100, 200, 300]
for c, a in zip(cities, amounts):
print(c, a)                    # 北京 100 / 上海 200 / 广州 300
```

- （别用 for i in range(len(cities)) 再 cities[i] —— 能跑，但 enumerate 更短更不容易越界。）

代码速查：今天这几段，要在本机跑通（1/5）

```
# 1) 分档：把 0-100 的分数映射成 A/B/C/D，用 for 把 55 72 88 96 全部判一遍
```

```
for score in [55, 72, 88, 96]:
```

```
    if score >= 90:
        grade = 'A'
    elif score >= 75:
        grade = 'B'
    elif score >= 60:
        grade = 'C'
    else:
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（2/5）

```
    grade = 'D'
print(score, grade)
# 2) 九九乘法表（用 for 嵌套，输出要能看）
for i in range(1, 10):
    for j in range(1, i + 1):
        print(f'{j}×{i}={i*j}', end='\t')
    print()
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（3/5）

```
# 3) 猜数字（while + break）：随机数 1-100，最多猜 7 次
```

```
import random
```

target = random.randint(1, 100)

```
for attempt in range(1, 8):
```

guess = int(input('猜一个 1-100 的数：'))

if guess == target:

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（4/5）

```
        print(f'第 {attempt} 次猜中，答案 {target}')
```

break

```
    print('小了' if guess < target else '大了')
```

```
else:
```

print(f'7 次没猜中，答案是 {target}')

验收（做到才算过关）：

① 九九乘法表的对齐是对的（用 \t 或 f'{...:2}'）；

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（5/5）

② 能说清 break 和 continue 的差别，各举一个今天代码里的例子；

③ 把猜数字改成「不限次数、直到猜中」，只改 while 条件，不许改别的。

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

最容易错的地方

★ 顺序写反就全错：如果把 >= 60 放在最前面，100 分也会被判成 C。

★ for-else 这个语法（循环没被 break 打断才执行 else）Python 有，但可读性差 ——面试写代码时别用，评委要反应两秒。

★ 推导式只能放表达式，不能放 print、不能放赋值。要写两句以上就用普通 for 循环，硬塞进推导式只会让代码更难读（也更容易写错）。

（别用 for i in range(len(cities)) 再 cities[i] —— 能跑，但 enumerate 更短更不容易越界。）

这些句子都是「差一个字、结果差一截」的地方，考前只翻这一页也值。

合上视频，自测一遍（能打勾才算学会）

- [ ] 我能不看资料写出：和 SQL 的 AND/OR 优先级规则一模一样 —— 这条你已经会了，只是换个写法。
- [ ] 我能不看资料写出：忘了 n += 1 就是死循环 —— 按 Ctrl+C 停。VSCode 里用「运行和调试」面板跑，比在终端里跑更好中断。
- [ ] 我能不看资料写出：for-else 这个语法（循环没被 break 打断才执行 else）Python 有，但可读性差 ——面试写代码时别用，评委要反应两秒。
- [ ] 我能不看资料写出：推导式只能放表达式，不能放 print、不能放赋值。要写两句以上就用普通 for 循环，硬塞进推导式只会让代码更难读（也更容易写错）。
- [ ] 我能不看题源，把今天用到的每个函数「为什么要有它」说一遍
- [ ] 我能说出今天每个参数/返回值改了会怎样——换个写法结果差在哪

这些句子都是「差一个字、结果差一截」的地方；三个勾以上，今天就算过关。

今天收工前，先知道明天学什么

第 3 天 • 容器四件套：list / tuple / dict / set，切片与排序

不用提前学。明天点「① 开始今天的学习」，会换上明天那份 PPT。

收工三件事（5 分钟）

- 今天没跑通的脚本存进 90-我的作业\，命名 2026-11-03-第02天-*.py
- 打卡表「③ 每日打卡」记一笔：今天卡在哪、错在哪（一句话就够）
- 错题明天重做一遍，练习数据在 60-练习环境\Python-Env\data

基础期 · 第 3 天 · 11/4

容器四件套：list tuple dict set，切片与排序

能用 list/dict 存起来，能按条件筛出子集并排序。

第 3 天 · 11/4 · 今日知识点回顾

容器四件套：list / tuple / dict / set，切片与排序

目标：拿到一份 CSV 的列名和几行数据，能用 list/dict 存起来，能按条件筛出子集并排序。

对着 B 站视频看：视频讲到的每一句，这张 PPT 里都有一条；复习时只翻这份，不用再回去翻分P。

今天结束时，你应该能做到

验收标准（做到这句就算过关）

- 拿到一份 CSV 的列名和几行数据，能用 list/dict 存起来，能按条件筛出子集并排序。

今天的做法（顺序别颠倒）

- 本机 VSCode 里写这三段：
- 题库地址：file:///D:/DSH工作区/00-学习中心/60-练习环境/Python-Env/data
- 做完记一句：今天卡在哪、错在哪 → 写进 3-每日打卡。

今天看哪几集、读哪一章

这一天的知识：5 集 • 合计约 82 分钟（边看边在本机敲一遍）

P62	数据容器入门	约 5 分钟
P63	列表的定义语法	约 11 分钟
P65	列表的常用操作方法	约 30 分钟
P71	数据容器（序列）的切片	约 13 分钟
P75	字典的定义	约 23 分钟

教材（已下载到 20-教材与讲义\python-data\，断网也能看） https://raw.githubusercontent.com/jackfrued/Python-100-Days/master/Day01-20/08.常用数据结构之列表-1.md

视频地址就在下面的「今天做哪些题」网页里，点一下直达分P。

知识点 1/7 • list：有序、可变、什么都能装

- 一句话：列表用方括号，能追加、能改、能排序，下标从 0 开始。
- 代码：

```
rows = []
rows.append(['100001', '北京', '1899.00'])
rows.append(['100002', '上海', '699.00'])
print(len(rows))          # 2
print(rows[0])            # ['100001', '北京', '1899.00']
print(rows[-1])           # ['100002', '上海', '699.00']  ← 负下标从尾巴数
```

知识点 1/7 • list：有序、可变、什么都能装（续）

- rows[0][1] = '杭州'

```
print(rows[0][1])          # 杭州
```

★ 二维列表就是「表的行」：读 CSV 时最常见的结构就是 list[list[str]]。

- 但它没有列名 —— 想按列名取值就得用 dict（知识点 5）。

知识点 2/7 • 切片：左闭右开，第三个参数是步长

- 一句话：lst[a:b:c] 从 a 取到 b（不含 b），步长 c；省略就取到头。
- 代码：

```
nums = list(range(10))       # [0,1,2,3,4,5,6,7,8,9]
print(nums[2:5])             # [2, 3, 4]
print(nums[:3])              # [0, 1, 2]
print(nums[-3:])             # [7, 8, 9]
print(nums[::2])             # [0, 2, 4, 6, 8]
print(nums[::-1])            # 倒序
```

★ 切片永远不报 IndexError（越界自动截断），而 nums[99] 会报。

- 「取前 N 行」写 rows[:5] 最安全，写 rows[0:5] 也行，但别写 rows[5]。

知识点 3/7 • list 的常用方法与排序

- 一句话：append/pop/sort/reverse 是原地改；sorted 是返回新表、不改原表。
- 代码：

```
a = [3, 1, 2]
b = sorted(a)                  # 新表，a 不变
print(a, b)                    # [3, 1, 2] [1, 2, 3]
a.sort(reverse=True)           # 原地排，a 变了，返回 None
print(a)                       # [3, 2, 1]
rows = [['A', 1899.0], ['B', 699.0], ['C', 1299.0]]
rows.sort(key=lambda r: r[1], reverse=True)
print(rows)                    # 按第 2 列金额降序
```

知识点 3/7 • list 的常用方法与排序（续）

★ a = a.sort() 是最常见的错：sort() 返回 None，赋值完 a 就成了 None，

- 下一行 a[0] 直接 TypeError: 'NoneType' object is not subscriptable。

知识点 4/7 • tuple：不可变，用来「一次返回多个值」

- 一句话：元组用圆括号，建好就不能改；函数返回多值时它就是元组。
- 代码：

```
def min_max(nums):
return min(nums), max(nums)      # 返回的是元组
```

这一页只讲「tuple：不可变，用来「一次返回多个值」」。讲完就翻页，下一页是另一个知识点。

知识点 4/7 • tuple：不可变，用来「一次返回多个值」（续）

- lo, hi = min_max([3, 1, 4, 1, 5])　# 拆包

```
print(lo, hi)                       # 1 5
point = (3, 4)
# point[0] = 9                      # TypeError: 'tuple' object does not support item assignment
```

★ 只有一个元素的元组要写 (5,) —— 写 (5) 就是整数 5，不是元组。这个坑在给函数传「单参数元组」时会静默出错。

知识点 5/7 • dict：按名字取值，不用记下标

- 一句话：字典是「键 → 值」的映射，用 d['键'] 取，用 d.get('键', 默认值) 防 KeyError。
- 代码：

```
row = {'order_id': '100001', 'city': '北京', 'amount': '1899.00'}
print(row['city'])                          # 北京
print(row.get('status', '（无此列）'))      # （无此列）  ← 不报错
print(list(row.keys()))                     # ['order_id', 'city', 'amount']
for k, v in row.items():
print(k, '=', v)
```

★ row['status'] 在键不存在时抛 KeyError: 'status'。读脏数据时列名常常对不上（多个空格、BOM、大小写），一律先 .get() 再判断。

知识点 6/7 • set：去重 + 集合运算

- 一句话：集合自动去重、没有顺序，能做交集/并集/差集。
- 代码：

```
cities = ['北京', '上海', '北京', '杭州', '上海']
print(set(cities))                      # {'北京', '上海', '杭州'}
print(len(set(cities)))                 # 3
```

这一页只讲「set：去重 + 集合运算」。讲完就翻页，下一页是另一个知识点。

知识点 6/7 • set：去重 + 集合运算（续）

- a, b = {'北京', '上海'}, {'上海', '广州'}

```
print(a & b)                    # 交集 {'上海'}
print(a | b)                    # 并集
print(a - b)                    # 差集 {'北京'}
```

- （本机 orders_dirty.csv 的 city 列去掉空值后有 9 个不同城市 ——用 len(set(...)) 就能验，第 9 天用 pandas 的 nunique() 会得到同一个数。）

知识点 7/7 • 把 CSV 读成「字典的列表」——清洗脚本的标准骨架

- 一句话：第一行当表头，后面每行按位置配成 {列名: 值}，这是 pandas 之前的通用写法。
- 代码：

```
import csv, pathlib
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
with open(DATA / 'orders_dirty.csv', encoding='utf-8-sig', newline='') as f:
reader = csv.DictReader(f)
rows = list(reader)
print(len(rows), rows[0]['order_id'], rows[0]['city'])
```

★ 用 csv 模块而不是 split(',')：商品名里有逗号（'三只松鼠坚果礼盒, 大包装'）时split 会把一列劈成两列，整行错位；csv.DictReader 会正确处理引号包裹。

代码速查：今天这几段，要在本机跑通（1/5）

```
import csv, pathlib
```

```
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
```

```
with open(DATA / 'orders_dirty.csv', encoding='utf-8-sig', newline='') as f:
```

rows = list(csv.DictReader(f))

1) 一共几行、列名有哪些

```
print(len(rows))
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（2/5）

```
print(list(rows[0].keys()))
```

```
# 2) 用 list 推导式筛出 city == '北京' 的行，取前 3 行打印
```

```
bj = [r for r in rows if r['city'] == '北京']
```

```
print(len(bj))
```

```
for r in bj[:3]:
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（3/5）

```
    print(r['order_id'], r['product_name'], r['amount'])
```

3) 用 set 统计有多少个不同城市（去掉空值）

```
cities = {r['city'] for r in rows if r['city']}
```

```
print(len(cities), sorted(cities))
```

4) 按金额降序排前 5（注意 amount 里有 '¥' 和逗号，先洗）

```
def to_num(s):
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（4/5）

```
    return float(s.replace('¥', '').replace(',', '')) if s else 0.0
```

```
top5 = sorted(rows, key=lambda r: to_num(r['amount']), reverse=True)[:5]
```

```
for r in top5:
```

print(r['order_id'], r['amount'])

验收（做到才算过关）：

① 第 3 步打印出的城市个数是 9；

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（5/5）

```
② 第 4 步能正确排出前 5 名（'¥1,899.00' 这种不能被当成字符串比大小）；
```

③ 能说清为什么「按字符串排序」和「按数字排序」结果不同。

最容易错的地方

「取前 N 行」写 rows[:5] 最安全，写 rows[0:5] 也行，但别写 rows[5]。

★ a = a.sort() 是最常见的错：sort() 返回 None，赋值完 a 就成了 None，

★ 只有一个元素的元组要写 (5,) —— 写 (5) 就是整数 5，不是元组。这个坑在给函数传「单参数元组」时会静默出错。

★ 用 csv 模块而不是 split(',')：商品名里有逗号（'三只松鼠坚果礼盒, 大包装'）时split 会把一列劈成两列，整行错位；csv.DictReader 会正确处理引号包裹。

这些句子都是「差一个字、结果差一截」的地方，考前只翻这一页也值。

合上视频，自测一遍（能打勾才算学会）

- [ ] 我能不看资料写出：切片永远不报 IndexError（越界自动截断），而 nums[99] 会报。
- [ ] 我能不看资料写出：只有一个元素的元组要写 (5,) —— 写 (5) 就是整数 5，不是元组。这个坑在给函数传「单参数元组」时会静默出错。
- [ ] 我能不看资料说出：「取前 N 行」写 rows[:5] 最安全，写 rows[0:5] 也行，但别写 rows[5]。
- [ ] 我能不看资料说出：一句话：集合自动去重、没有顺序，能做交集/并集/差集。
- [ ] 我能不看题源，把今天用到的每个函数「为什么要有它」说一遍
- [ ] 我能说出今天每个参数/返回值改了会怎样——换个写法结果差在哪

这些句子都是「差一个字、结果差一截」的地方；三个勾以上，今天就算过关。

今天收工前，先知道明天学什么

第 4 天 • ★Excel 一天速成：数据透视表 + XLOOKUP/VLOOKUP + SUMIFS

不用提前学。明天点「① 开始今天的学习」，会换上明天那份 PPT。

收工三件事（5 分钟）

- 今天没跑通的脚本存进 90-我的作业\，命名 2026-11-04-第03天-*.py
- 打卡表「③ 每日打卡」记一笔：今天卡在哪、错在哪（一句话就够）
- 错题明天重做一遍，练习数据在 60-练习环境\Python-Env\data

基础期 • 第 4 天 • 11/5

★Excel 一天速成：数据透视表 + XLOOKUPVLOOKUP + SUMIFS

能说清每一步在干什么。

第 4 天 · 11/5 · 今日知识点回顾

★Excel 一天速成：数据透视表 + XLOOKUP/VLOOKUP + SUMIFS

目标：15 分钟内把一张脏表做成透视汇总表 —— 透视表 + 至少一个查找函数，且能说清每一步在干什么。

对着 B 站视频看：视频讲到的每一句，这张 PPT 里都有一条；复习时只翻这份，不用再回去翻分P。

今天结束时，你应该能做到

验收标准（做到这句就算过关）

- 15 分钟内把一张脏表做成透视汇总表 —— 透视表 + 至少一个查找函数，且能说清每一步在干什么。

今天的做法（顺序别颠倒）

- 题库地址：file:///D:/DSH工作区/00-学习中心/60-练习环境/Python-Env/data
- （今天的练习表是 sales_dirty.xlsx；第 20 天还有一天 Excel 进阶，学 Power Query）
- 做完记一句：今天卡在哪、错在哪 → 写进 3-每日打卡。

今天看哪几集、读哪一章

这一天的知识：5 集 • 合计约 62 分钟（边看边在本机敲一遍）

P44	数据透视表的创建	约 15 分钟
P45	数据透视表的结构	约 12 分钟
P49	数据透视表的值汇总方式	约 12 分钟
P28	函数-SUM函数	约 8 分钟
P30	函数-SUMIFS函数	约 15 分钟

教材（已下载到 20-教材与讲义\python-data\，断网也能看） https://support.microsoft.com/zh-cn/office/创建数据透视表以分析工作表数据-0b9a4a4e-1a0b-4d6e-8c4e-1a1b1c1d1e1f

视频地址就在下面的「今天做哪些题」网页里，点一下直达分P。

知识点 1/7 • 透视表只做三件事：分组、汇总、交叉

- 一句话：行 = 按什么分组，列 = 按什么拆开，值 = 算什么（求和/计数/平均）。
- 操作：
- 选中数据区域任意单元格 → 插入 → 数据透视表 → 确定把「品类」拖到【行】，把「城市」拖到【列】，把「金额」拖到【值】

★ 透视表不会自动刷新：源数据改了必须右键 → 刷新（或 Alt+F5）。

- 忘了刷新是「数字对不上」最常见的原因，而且它不报错。

知识点 2/7 • 值汇总方式决定「求和还是计数」

- 一句话：默认是「求和」，但源列里有文本时 Excel 会自动变成「计数」。
- 操作：
- 值区域 → 右键 → 值字段设置 → 求和 / 计数 / 平均值 / 最大值

★ 金额列里混了 '¥1,899.00' 这种文本，透视表会把它算成 0 或直接计数 ——进透视表之前必须先把这列清成纯数字（分列 / 查找替换 / VALUE 函数）。

知识点 3/7 • XLOOKUP：查找函数的当前版本

- 一句话：=XLOOKUP(找什么, 在哪一列找, 返回哪一列, 找不到时返回什么)。
- 操作：
- =XLOOKUP(A2, 商品表!A:A, 商品表!D:D, "未找到")
- =XLOOKUP(A2&B2, 商品表!A:A&商品表!B:B, 商品表!D:D, "未找到") ← 多条件查找

★ XLOOKUP 的第 2、3 参数是「整列」，不用数第几列 —— 这是它比 VLOOKUP 强的核心。

- VLOOKUP 只能往右找（返回列必须在查找列右边），XLOOKUP 往左往右都行。

知识点 4/7 • VLOOKUP 仍然要会，因为面试还在问

- 一句话：=VLOOKUP(找什么, 区域, 返回第几列, 0)，最后的 0 表示精确匹配。
- 操作：
- =VLOOKUP(A2, 商品表!A:D, 4, 0)

★ 省略第 4 参数（写成 =VLOOKUP(A2,商品表!A:D,4)）是模糊匹配，会返回一个「看起来像但不对」的值，且不报错 —— 这是 Excel 最经典的静默错误。

知识点 5/7 • SUMIFS：多条件求和，参数顺序是「先求和列」

- 一句话：=SUMIFS(求和列, 条件列1, 条件1, 条件列2, 条件2, ...)。
- 操作：
- =SUMIFS(明细!H:H, 明细!C:C, "北京", 明细!D:D, "数码")

★ SUMIFS 和 SUMIF 的参数顺序相反**：SUMIF 是（条件列, 条件, 求和列），SUMIFS 是（求和列, 条件列, 条件）。抄错一次就会算出 0。**

知识点 6/7 • 条件格式：把「异常」标出来

- 一句话：开始 → 条件格式 → 突出显示单元格规则 / 数据条 / 色阶。
- 操作：
- 选中金额列 → 条件格式 → 数据条（一眼看出谁高谁低）
- 选中金额列 → 条件格式 → 突出显示 → 大于 → 5000 → 红色填充

★ 条件格式跟着「单元格值」走，不跟着「行」走。筛选后隐藏的行仍会被染色，打印前记得检查。

知识点 7/7 • 15 分钟做完的固定动作清单

- 一句话：清洗 → 透视 → 查找 → 条件格式，四步，每步都有时间预算。
- 操作：
- ① 0–3 分钟：全选 → 删除重复项；把金额列分列成数字；日期列统一格式② 3–8 分钟：插入透视表（行=品类，列=城市，值=金额求和）
- ③ 8–12 分钟：在旁边用 XLOOKUP 把「大区」贴进来④ 12–15 分钟：条件格式标出 Top 10%，加一行总计

- （本机 Excel 2026 已装：C:\Program Files\Microsoft Office\root\Office16\EXCEL.EXE。
- 今天的练习表在 data\sales_dirty.xlsx，3 个 sheet：明细 400 行 / 商品表 15 行 / 城市表 9 行。）

代码速查：今天这几段，要在本机跑通（1/2）

打开 data\sales_dirty.xlsx，按下面顺序做，全程计时 15 分钟：

① 明细 sheet：删除重复订单号 → 金额列分列成数字 → 日期列统一成 yyyy-mm-dd

② 插入透视表：行=品类、列=城市、值=金额求和，放到新 sheet 命名「汇总」

③ 在「汇总」右侧用 XLOOKUP 把城市表里的大区贴进来（城市是键）

④ 用 SUMIFS 在明细下方算出「北京 + 数码」的合计金额，和透视表对应格子核对

⑤ 条件格式：金额列加数据条；大于 5000 的标红

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（2/2）

验收（做到才算过关）：

① 15 分钟内做出「汇总」表，行列都对得上；

② XLOOKUP 和 SUMIFS 各写出一条公式，并能讲清每个参数是什么；

③ 能说出透视表不刷新会导致什么后果（举一个具体场景）。

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

最容易错的地方

忘了刷新是「数字对不上」最常见的原因，而且它不报错。

★ 省略第 4 参数（写成 =VLOOKUP(A2,商品表!A:D,4)）是模糊匹配，会返回一个「看起来像但不对」的值，且不报错 —— 这是 Excel 最经典的静默错误。

★ SUMIFS 和 SUMIF 的参数顺序**相反**：SUMIF 是（条件列, 条件, 求和列），SUMIFS 是（求和列, 条件列, 条件）。抄错一次就会算出 0。

这些句子都是「差一个字、结果差一截」的地方，考前只翻这一页也值。

合上视频，自测一遍（能打勾才算学会）

- [ ] 我能不看资料写出：XLOOKUP 的第 2、3 参数是「整列」，不用数第几列 —— 这是它比 VLOOKUP 强的核心。
- [ ] 我能不看资料写出：条件格式跟着「单元格值」走，不跟着「行」走。筛选后隐藏的行仍会被染色，打印前记得检查。
- [ ] 我能不看资料说出：VLOOKUP 只能往右找（返回列必须在查找列右边），XLOOKUP 往左往右都行。
- [ ] 我能不看题源，把今天用到的每个函数「为什么要有它」说一遍
- [ ] 我能说出今天每个参数/返回值改了会怎样——换个写法结果差在哪
- [ ] 今天这几段代码我在本机都跑通、并看清了输出

这些句子都是「差一个字、结果差一截」的地方；三个勾以上，今天就算过关。

今天收工前，先知道明天学什么

第 5 天 • 函数：参数、返回值、作用域、lambda

不用提前学。明天点「① 开始今天的学习」，会换上明天那份 PPT。

收工三件事（5 分钟）

- 今天没跑通的脚本存进 90-我的作业\，命名 2026-11-05-第04天-*.py
- 打卡表「③ 每日打卡」记一笔：今天卡在哪、错在哪（一句话就够）
- 错题明天重做一遍，练习数据在 60-练习环境\Python-Env\data

基础期 • 第 5 天 • 11/6

函数：参数、返回值、作用域、lambda

能说清参数是「值传递」还是「引用传递」。

第 5 天 · 11/6 · 今日知识点回顾

函数：参数、返回值、作用域、lambda

目标：把一段 30 行的流水账脚本拆成 4 个函数，每个函数只干一件事，且能说清参数是「值传递」还是「引用传递」。

对着 B 站视频看：视频讲到的每一句，这张 PPT 里都有一条；复习时只翻这份，不用再回去翻分P。

今天结束时，你应该能做到

验收标准（做到这句就算过关）

- 把一段 30 行的流水账脚本拆成 4 个函数，每个函数只干一件事，且能说清参数是「值传递」还是「引用传递」。

今天的做法（顺序别颠倒）

- 题库地址：file:///D:/DSH工作区/00-学习中心/60-练习环境/Python-Env/data
- 做完记一句：今天卡在哪、错在哪 → 写进 3-每日打卡。

今天看哪几集、读哪一章

这一天的知识：6 集 · 合计约 58 分钟（边看边在本机敲一遍）

P51	函数的初体验	约 12 分钟
P52	函数的基础定义语法	约 9 分钟
P54	函数的传入参数	约 10 分钟
P56	函数的返回值定义语法	约 8 分钟
P60	变量在函数中的作用域	约 10 分钟
P84	lambda匿名函数	约 9 分钟

教材（已下载到 20-教材与讲义\python-data\，断网也能看） https://raw.githubusercontent.com/jackfrued/Python-100-Days/master/Day01-20/14.函数和模块.md

视频地址就在下面的「今天做哪些题」网页里，点一下直达分P。

知识点 1/6 • def 定义，return 返回

- 一句话：def 名字(参数): 下面缩进写函数体；return 把结果交出去，没写 return 就返回 None。
- 代码：

```
def bmi(height_m, weight_kg):
return weight_kg / height_m ** 2
print(bmi(1.75, 70))        # 22.857142857142858
print(round(bmi(1.75, 70), 2))   # 22.86
print(bmi(weight_kg=70, height_m=1.75))   # 22.857... ← 关键字参数，顺序无所谓
```

★ 忘了 return 是最隐蔽的 bug：函数体里 print 了结果，但函数本身返回 None，外面 a = f() 之后 a 就是 None，后面 a + 1 报 TypeError。

知识点 2/6 • 默认参数与可变参数

- 一句话：默认参数写 def f(a, b=10)；可变参数写 *args（元组）和 **kwargs（字典）。
- 代码：

```
def to_num(s, default=0.0):
try:
return float(str(s).replace('¥', '').replace(',', ''))
except (TypeError, ValueError):
return default
print(to_num('¥1,899.00'))   # 1899.0
print(to_num(''))            # 0.0
print(to_num(None, -1))      # -1
```

知识点 2/6 • 默认参数与可变参数（续）

★ 默认参数只在「定义时」求值一次。def f(x, bucket=[]) 里的 bucket 会被所有调用

- 共享，追加几次就串了 —— 要用 None 兜：def f(x, bucket=None): bucket = bucket or []

知识点 3/6 • 作用域：函数里改不了外面的数字，但能改列表

- 一句话：函数内赋值会新建局部变量；global 才能改全局，但尽量别用。
- 代码：

```
counter = 0
def bump():
global counter
```

- counter += 1bump(); bump()

知识点 3/6 • 作用域：函数里改不了外面的数字，但能改列表（续）

```
print(counter)          # 2
nums = [1, 2, 3]
def add(x):
x.append(4)         # 没有赋值，直接改对象 → 外面也变
add(nums)
print(nums)             # [1, 2, 3, 4]
```

★ 「值传递 / 引用传递」这个面试题的正确答法：Python 传的是「对象引用」。

- 函数内重新赋值不影响外面，原地修改（append / sort / [0]=）会影响外面。

知识点 4/6 • lambda：一行的匿名函数，专门喂给 key

- 一句话：lambda 参数: 表达式，通常只用在 sorted / max / min / map / filter 的 key 上。
- 代码：

```
rows = [{'city': '北京', 'amount': 1899.0},
```

- {'city': '上海', 'amount': 699.0}]rows.sort(key=lambda r: r['amount'], reverse=True)

知识点 4/6 • lambda：一行的匿名函数，专门喂给 key（续）

```
print([r['city'] for r in rows])          # ['北京', '上海']
print(max(rows, key=lambda r: r['amount'])['city'])   # 北京
```

> **★ lambda 里不能写语句（不能 print、不能赋值、不能 if-else 的多分支），只能是一个表达式。超过一行的逻辑就老老实实写 def。**

知识点 5/6 • map / filter / 列表推导式，选一个就行

- 一句话：三者都能做转换和筛选，团队代码里推导式最易读，map/filter 次之。
- 代码：

```
nums = [1, 2, 3, 4, 5]
print([n * 2 for n in nums])                # 推导式（推荐）
print(list(map(lambda n: n * 2, nums)))     # map
print([n for n in nums if n % 2 == 0])      # 筛选
print(list(filter(lambda n: n % 2 == 0, nums)))
```

★ map/filter 返回的是迭代器，不是列表 —— 直接 print 会得到 <map object at 0x...>，要么套 list()，要么改成推导式。这个坑在 Python 3 里天天有人踩。

知识点 6/6 • 写函数的三条硬规矩

- 一句话：一个函数只干一件事；参数不超过 4 个；有副作用（读写文件/打印）要写进名字。
- 代码：

```
def load_orders(path): ...          # 读文件
def clean_amount(rows): ...         # 清洗
def summarize_by_city(rows): ...    # 汇总
def save_csv(rows, path): ...       # 写出
```

- （这四步就是第 14 天 W5 验收脚本的骨架。今天先按这个结构把昨天的练习拆成函数，明天开始每个函数单独能测。）

代码速查：今天这几段，要在本机跑通（1/5）

把第 3 天的练习脚本重写成 4 个函数，每个函数单独能跑：

```
import csv, pathlib
```

```
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
```

```
def load_orders(path):
```

```
    with open(path, encoding='utf-8-sig', newline='') as f:
```

return list(csv.DictReader(f))

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（2/5）

```
def to_num(s, default=0.0):
```

```
    try:
```

```
        return float(str(s).replace('¥', '').replace(',', ''))
```

```
    except (TypeError, ValueError):
```

```
        return default
```

```
def summarize_by_city(rows, status='paid'):
```

```
    agg = {}
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（3/5）

for r in rows:

```
        if r['status'].strip().lower() != status:
```

continue

city = r['city'] or '（未知）'

```
        agg[city] = agg.get(city, 0.0) + to_num(r['amount'])
```

```
    return dict(sorted(agg.items(), key=lambda kv: kv[1], reverse=True))
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（4/5）

```
if __name__ == '__main__':
```

```
    rows = load_orders(DATA / 'orders_dirty.csv')
```

print(len(rows), '行')

```
    for city, total in list(summarize_by_city(rows).items())[:5]:
```

print(f'{city}\t{total:,.2f}')

验收（做到才算过关）：

① 四个函数各自能单独调用、单独验证；

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（5/5）

② `if __name__ == '__main__':` 那一行的作用能讲清楚（模块被 import 时不执行）；

③ 把 summarize_by_city 改成按 category 汇总，只改一行参数。

最容易错的地方

共享，追加几次就串了 —— 要用 None 兜：def f(x, bucket=None): bucket = bucket or []◆ 知识点 3 | 作用域：函数里改不了外面的数字，但能改列表一句话：函数内赋值会新建局部变量；global 才能改全局，但尽量别用。

这些句子都是「差一个字、结果差一截」的地方，考前只翻这一页也值。

合上视频，自测一遍（能打勾才算学会）

- [ ] 我能不看资料写出：lambda 里不能写语句（不能 print、不能赋值、不能 if-else 的多分支），只能是一个表达式。超过一行的逻辑就老老实实写 def。
- [ ] 我能不看题源，把今天用到的每个函数「为什么要有它」说一遍
- [ ] 我能说出今天每个参数/返回值改了会怎样——换个写法结果差在哪
- [ ] 今天这几段代码我在本机都跑通、并看清了输出

这些句子都是「差一个字、结果差一截」的地方；三个勾以上，今天就算过关。

今天收工前，先知道明天学什么

第 6 天 • 面向对象 + 模块与包 + 常用标准库

不用提前学。明天点「① 开始今天的学习」，会换上明天那份 PPT。

收工三件事（5 分钟）

- 今天没跑通的脚本存进 90-我的作业\，命名 2026-11-06-第05天-*.py
- 打卡表「③ 每日打卡」记一笔：今天卡在哪、错在哪（一句话就够）
- 错题明天重做一遍，练习数据在 60-练习环境\Python-Env\data

基础期 • 第 6 天 • 11/7

面向对象 + 模块与包 + 常用标准库

能说清 import 的三种写法差别。

第 6 天 · 11/7 · 今日知识点回顾

面向对象 + 模块与包 + 常用标准库

目标：写一个带 __init__ 和 __repr__ 的类，把昨天的 4 个函数收进一个类里；能说清 import 的三种写法差别。

对着 B 站视频看：视频讲到的每一句，这张 PPT 里都有一条；复习时只翻这份，不用再回去翻分P。

今天结束时，你应该能做到

验收标准（做到这句就算过关）

- 写一个带 __init__ 和 __repr__ 的类，把昨天的 4 个函数收进一个类里；能说清 import 的三种写法差别。

今天的做法（顺序别颠倒）

- 题库地址：file:///D:/DSH工作区/00-学习中心/60-练习环境/Python-Env/data
- 做完记一句：今天卡在哪、错在哪 → 写进 3-每日打卡。

今天看哪几集、读哪一章

这一天的知识：5 集 • 合计约 73 分钟（边看边在本机敲一遍）

P111	初识对象	约 10 分钟
P112	类的成员方法	约 18 分钟
P114	构造方法	约 12 分钟
P115	魔术方法	约 15 分钟
P94	模块的概念和导入	约 18 分钟

教材（已下载到 20-教材与讲义\python-data\，断网也能看） https://raw.githubusercontent.com/jackfrued/Python-100-Days/master/Day01-20/18.面向对象编程入门.md

视频地址就在下面的「今天做哪些题」网页里，点一下直达分P。

知识点 1/7 • 类 = 模板，对象 = 按模板造出来的东西

- 一句话：class 定义模板，__init__ 在造对象时自动跑，self 指「这个对象自己」。
- 代码：

```
class Order:
def __init__(self, order_id, city, amount):
```

- self.order_id = order_idself.city = cityself.amount = amount

这一页只讲「类 = 模板，对象 = 按模板造出来的东西」。讲完就翻页，下一页是另一个知识点。

知识点 1/7 • 类 = 模板，对象 = 按模板造出来的东西（续）

```
o = Order('100001', '北京', 1899.0)
print(o.city)          # 北京
print(o.amount)        # 1899.0
```

★ self 不能省，也不能换成别的名字（虽然语法允许）。类里每个方法的第一个参数都是 self，调用时不传 —— o.city 会自动变成 Order.city(o)。

知识点 2/7 • __repr__ 决定「print 出来长什么样」

- 一句话：没写 __repr__ 时 print(o) 打印 <__main__.Order object at 0x...>，看不出内容。
- 代码：

```
class Order:
def __init__(self, order_id, city, amount):
```

- self.order_id, self.city, self.amount = order_id, city, amount

知识点 2/7 • __repr__ 决定「print 出来长什么样」（续）

```
def __repr__(self):
return f'Order({self.order_id}, {self.city}, {self.amount:.2f})'
print(Order('100001', '北京', 1899.0))     # Order(100001, 北京, 1899.00)
```

★ 调试时 90% 的时间花在「看对象里有什么」。__repr__ 写好了，print 一次就够；

- 不写，就得 print(o.__dict__) 或进调试器。

知识点 3/7 • 方法：把「针对这个对象的操作」写进类里

- 一句话：方法就是第一个参数是 self 的函数，能直接访问 self.xxx。
- 代码：

```
class Order:
def __init__(self, order_id, city, amount):
```

- self.order_id, self.city, self.amount = order_id, city, amount

知识点 3/7 • 方法：把「针对这个对象的操作」写进类里（续）

```
def is_big(self, threshold=1000):
return self.amount >= threshold
def discounted(self, rate=0.9):
return round(self.amount * rate, 2)
o = Order('100001', '北京', 1899.0)
print(o.is_big(), o.discounted())      # True 1709.1
```

★ 类里的 amount 用 float 存，别存 '¥1,899.00' 字符串 —— 清洗放在构造之前（第 5 天的 to_num 就是干这个的），类里只放干净数据。

知识点 4/7 • 继承：把公共部分提到父类

- 一句话：class 子类(父类): 自动拥有父类的属性和方法，需要改就覆写。
- 代码：

```
class Base:
def describe(self):
return f'{type(self).__name__}: {self.__dict__}'
class PaidOrder(Base):
def __init__(self, oid, amount):
```

知识点 4/7 • 继承：把公共部分提到父类（续）

- self.oid, self.amount = oid, amount

```
print(PaidOrder('100001', 1899.0).describe())
```

★ 数据分析岗的代码里继承用得很少 —— 别为了「用上继承」而硬造类层级。

- 真正高频的是「一个类装一份数据 + 几个方法算指标」，今天写出来就够。

知识点 5/7 • import 的三种写法，差别在「名字进不进当前命名空间」

- 一句话：import x / from x import y / import x as z，优先用第一种或第三种。
- 代码：

```
import pathlib                  # 用 pathlib.Path
from pathlib import Path        # 直接用 Path
import numpy as np              # 惯例缩写
import pandas as pd
```

★ from module import * 会把模块里所有公开名字倒进来，容易和你的变量撞名，而且看不出某个函数是哪来的 —— 项目里禁止用（PEP 8 明确不建议）。

知识点 6/7 • 四个天天要用的标准库

- 一句话：pathlib 管路径、datetime 管日期、collections 管计数、json 管序列化。
- 代码：

```
import pathlib, datetime as dt, collections, json
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
print(DATA.exists(), (DATA / 'orders_dirty.csv').stat().st_size)
print(dt.date(2026, 11, 7).strftime('%Y-%m-%d %A'))
c = collections.Counter(['北京', '上海', '北京'])
print(c.most_common())            # [('北京', 2), ('上海', 1)]
print(json.dumps({'city': '北京'}, ensure_ascii=False))   # {"city": "北京"}
```

★ json.dumps 默认 ensure_ascii=True，中文会变成 \u5317\u4eac。写文件给人看时一定要 ensure_ascii=False，否则打开是一堆转义。

知识点 7/7 • 把函数收进类：什么时候值得

- 一句话：一份数据要反复算多个指标、或要带着状态（清洗前后的行数）时，才值得建类。
- 代码：

```
class Orders:
def __init__(self, rows):
```

- self.rows = rowsself.dropped = 0

```
def filter_paid(self):
before = len(self.rows)
```

知识点 7/7 • 把函数收进类：什么时候值得（续）

- self.rows = [r for r in self.rows if r['status'].strip().lower() == 'paid']self.dropped += before - len(self.rows)

```
return self
```

（self.dropped 这种「顺手记一笔」的状态，是类比纯函数好用的地方：

- 纯函数每次都要把计数传进去再传出来。）

代码速查：今天这几段，要在本机跑通（1/5）

把第 5 天的四个函数收进一个类，并加两个方法：

```
import csv, pathlib
```

```
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
```

```
class Orders:
```

```
    def __init__(self, rows):
        self.rows = rows
        self.log = []
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（2/5）

def __repr__(self):

```
        return f'Orders({len(self.rows)} 行，已丢弃 {sum(self.log) if self.log else 0})'
```

```
    def filter_status(self, status='paid'):
```

before = len(self.rows)

```
        self.rows = [r for r in self.rows if r['status'].strip().lower() == status]
```

```
        self.log.append(before - len(self.rows))
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（3/5）

```
        return self
    def sum_by(self, col='city'):
        agg = {}
        for r in self.rows:
            k = r[col] or '（未知）'
```

```
            v = float(r['amount'].replace('¥', '').replace(',', '') or 0)
```

```
            agg[k] = agg.get(k, 0.0) + v
```

```
        return dict(sorted(agg.items(), key=lambda kv: kv[1], reverse=True))
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（4/5）

```
if __name__ == '__main__':
```

```
    with open(DATA / 'orders_dirty.csv', encoding='utf-8-sig', newline='') as f:
```

```
    rows = list(csv.DictReader(f))
o = Orders(rows)
print(o)
print(o.filter_status('paid'))
top = o.sum_by('city')
for k, v in list(top.items())[:3]:
    print(f'{k}\t{v:,.2f}')
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（5/5）

验收（做到才算过关）：

```
① print(o) 打印的是 Orders(1200 行, 已丢弃 0)，不是 <object at 0x...>；
```

② filter_status 之后 print(o) 能看出丢了多少行；

```
③ 能说出「为什么 sum_by 里对 amount 又要洗一遍」— 因为第 3 天没洗，
```

原始 rows 里的 amount 还带着 ¥ 和逗号。

最容易错的地方

★ self 不能省，也不能换成别的名字（虽然语法允许）。类里每个方法的第一个参数都是 self，调用时不传 —— o.city 会自动变成 Order.city(o)。

★ 类里的 amount 用 float 存，别存 '¥1,899.00' 字符串 —— 清洗放在构造之前（第 5 天的 to_num 就是干这个的），类里只放干净数据。

★ 数据分析岗的代码里继承用得很少 —— 别为了「用上继承」而硬造类层级。

这些句子都是「差一个字、结果差一截」的地方，考前只翻这一页也值。

合上视频，自测一遍（能打勾才算学会）

- [ ] 我能不看资料写出：数据分析岗的代码里继承用得很少 —— 别为了「用上继承」而硬造类层级。
- [ ] 我能不看题源，把今天用到的每个函数「为什么要有它」说一遍
- [ ] 我能说出今天每个参数/返回值改了会怎样——换个写法结果差在哪
- [ ] 今天这几段代码我在本机都跑通、并看清了输出

这些句子都是「差一个字、结果差一截」的地方；三个勾以上，今天就算过关。

今天收工前，先知道明天学什么

第 7 天 • 文件 IO + 异常处理 + CSV/JSON 读写（W4 收尾验收）

不用提前学。明天点「① 开始今天的学习」，会换上明天那份 PPT。

收工三件事（5 分钟）

- 今天没跑通的脚本存进 90-我的作业\，命名 2026-11-07-第06天-*.py
- 打卡表「③ 每日打卡」记一笔：今天卡在哪、错在哪（一句话就够）
- 错题明天重做一遍，练习数据在 60-练习环境\Python-Env\data

基础期 • 第 7 天 • 11/8

文件 IO + 异常处理 + CSVJSON 读写（W4 收尾验收）

写一个脚本：读脏 CSV → 清洗 → 统计 → 写出干净 CSV + 一份 JSON 摘要，全程用 try/except 兜住，中途出错不崩。

第 7 天 · 11/8 · 今日知识点回顾

文件 IO + 异常处理 + CSV/JSON 读写（W4 收尾验收）

目标：写一个脚本：读脏 CSV → 清洗 → 统计 → 写出干净 CSV + 一份 JSON 摘要，全程用 try/except 兜住，中途出错不崩。

对着 B 站视频看：视频讲到的每一句，这张 PPT 里都有一条；复习时只翻这份，不用再回去翻分P。

今天结束时，你应该能做到

验收标准（做到这句就算过关）

- 写一个脚本：读脏 CSV → 清洗 → 统计 → 写出干净 CSV + 一份 JSON 摘要，全程用 try/except 兜住，中途出错不崩。

今天的做法（顺序别颠倒）

- 题库地址：file:///D:/DSH工作区/00-学习中心/60-练习环境/Python-Env/data
- 做完记一句：今天卡在哪、错在哪 → 写进 3-每日打卡。

今天看哪几集、读哪一章

这一天的知识：5 集 · 合计约 79 分钟（边看边在本机敲一遍）

P86	文件的读取操作	约 28 分钟
P88	文件的写出操作	约 10 分钟
P91	了解异常	约 5 分钟
P92	异常的捕获	约 21 分钟
P100	JSON数据格式的转换	约 15 分钟

教材（已下载到 20-教材与讲义\python-data\，断网也能看） https://raw.githubusercontent.com/jackfrued/Python-100-Days/master/Day21-30/21.文件读写和异常处理.md

视频地址就在下面的「今天做哪些题」网页里，点一下直达分P。

知识点 1/7 • with open 是唯一正确写法

- 一句话：with 会在块结束时自动关文件，不用手写 f.close()。
- 代码：

```
import pathlib
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
with open(DATA / 'orders_dirty.csv', encoding='utf-8-sig', newline='') as f:
head = f.readline()
print(head.strip()[:60])
```

★ 三个参数缺一不可：encoding='utf-8-sig' 吃掉 BOM；newline='' 让 csv 模块自己处理换行（不写的话 Windows 上会出现空行翻倍）。模式 'r' 读、'w' 覆盖写、'a' 追加。

知识点 2/7 • 'w' 会瞬间清空文件

- 一句话：open(path, 'w') 一打开就把原文件清成 0 字节，写失败也回不来。
- 代码：

```
out = DATA / '_tmp_test.csv'
with open(out, 'w', encoding='utf-8-sig', newline='') as f:
f.write('a,b\n1,2\n')
print(out.exists(), out.stat().st_size)     # True 8
out.unlink()                                 # 用完删掉
```

★ 往已有文件里补内容要用 'a'；要「读改写」就写新文件再替换，别直接 'w' 覆盖 ——这一步出错，原数据就没了。

知识点 3/7 • try/except：只兜你预期的错

- 一句话：except 后面写具体异常类型，兜不住的错误要让它照常报出来。
- 代码：

```
def to_num(s, default=0.0):
try:
return float(str(s).replace('¥', '').replace(',', ''))
except (TypeError, ValueError) as e:
return default
print(to_num('¥1,899.00'), to_num(''), to_num(None))
```

★ 裸 except: 会把 KeyboardInterrupt（Ctrl+C）和语法错误一起吞掉，程序该停不停 ——永远写具体类型，至少写 except Exception。

这一页只讲「try/except：只兜你预期的错」。讲完就翻页，下一页是另一个知识点。

知识点 4/7 • 文件不存在、编码不对，各报什么错

- 一句话：FileNotFoundError 是路径错；UnicodeDecodeError 是编码错；KeyError 是列名错。
- 代码：

```
try:
open(DATA / '不存在.csv', encoding='utf-8')
except FileNotFoundError as e:
print('路径错：', e)
try:
open(DATA / 'orders_dirty.csv', encoding='utf-8').readline()
except UnicodeDecodeError as e:
print('编码错，要加 -sig：', str(e)[:60])
```

知识点 4/7 • 文件不存在、编码不对，各报什么错（续）

- （本机实测：不带 utf-8-sig 读 orders_dirty.csv 会报
- UnicodeDecodeError: 'utf-8' codec can't decode byte 0xef in position 0 —— 0xef 就是 BOM 的 EF BB BF。）

知识点 5/7 • csv.DictWriter：按表头写出

- 一句话：先 writer.writeheader() 写表头，再 writer.writerow(dict) 一行行写。
- 代码：

```
import csv
out = DATA / '_clean_sample.csv'
with open(out, 'w', encoding='utf-8-sig', newline='') as f:
w = csv.DictWriter(f, fieldnames=['order_id', 'city', 'amount'])
w.writeheader()
w.writerows([{'order_id': '100001', 'city': '北京', 'amount': 1899.0}])
print(out.stat().st_size)
out.unlink()
```

知识点 5/7 • csv.DictWriter：按表头写出（续）

★ fieldnames 里的列名必须和 dict 的键完全一致，否则报 ValueError:

- dict contains fields not in fieldnames。多出来的键用 extrasaction='ignore' 忽略。

知识点 6/7 • JSON：把统计结果存成「人能读、程序能吃」的格式

- 一句话：json.dump(obj, f, ensure_ascii=False, indent=2)。
- 代码：

```
import json
summary = {'rows': 1200, 'paid_rows': 417, 'paid_sum': 422436.0,
```

- 'top_city': {'南京': 60532.0}}

知识点 6/7 • JSON：把统计结果存成「人能读、程序能吃」的格式（续）

```
with open(DATA / '_summary.json', 'w', encoding='utf-8') as f:
json.dump(summary, f, ensure_ascii=False, indent=2)
print((DATA / '_summary.json').read_text(encoding='utf-8')[:80])
(DATA / '_summary.json').unlink()
```

★ JSON 不支持 datetime / Decimal / numpy 类型 —— 直接 dump 会报

- TypeError: Object of type Timestamp is not JSON serializable。
- 先转成字符串或 float（后面 pandas 阶段天天遇到）。

知识点 7/7 • W4 验收脚本的完整骨架

- 一句话：load → clean → summarize → save，四段各一个函数，main 里只做编排。
- 代码：

```
def main():
rows = load_orders(DATA / 'orders_dirty.csv')      # 读
```

- clean, report = clean_orders(rows)   # 洗 + 记清洗账

知识点 7/7 • W4 验收脚本的完整骨架（续）

```
summary = summarize(clean)                          # 汇总
save_csv(clean, DATA / 'orders_clean.csv')          # 写干净数据
save_json({**report, **summary}, DATA / 'orders_summary.json')
print(f'清洗后 {len(clean)} 行，摘要已写出')
```

- （这份 orders_clean.csv / orders_summary.json 就是 W4 的交付物，W5 直接拿来用 ——今天写不好，后面 7 天都在脏数据上打转。）

代码速查：今天这几段，要在本机跑通（1/9）

写一个 orders_clean.py，跑出 orders_clean.csv + orders_summary.json：

```
import csv, json, pathlib
```

```
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
```

```
def to_num(s, default=None):
```

try:

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（2/9）

```
        return float(str(s).replace('¥', '').replace(',', ''))
```

except (TypeError, ValueError):

return default

```
def load_orders(path):
```

```
    with open(path, encoding='utf-8-sig', newline='') as f:
```

return list(csv.DictReader(f))

代码速查：今天这几段，要在本机跑通（3/9）

```
def clean_orders(rows):
```

```
    seen, clean = set(), []
```

```
    dropped = {'dup': 0, 'bad_amount': 0, 'bad_qty': 0}
```

```
    for r in rows:
        oid = r['order_id']
        if oid in seen:
            dropped['dup'] += 1
            continue
        seen.add(oid)
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（4/9）

```
amt = to_num(r['amount'])
if amt is None:
    dropped['bad_amount'] += 1
    continue
qty = to_num(r['quantity'])
if qty is None or qty <= 0:
    dropped['bad_qty'] += 1
    continue
r['amount'] = round(amt, 2)
r['quantity'] = int(qty)
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（5/9）

```
        r['status'] = (r['status'] or '').strip().lower()
```

```
        r['city'] = (r['city'] or '').strip() or '（未知）'
```

clean.append(r)

return clean, dropped

```
def summarize(rows):
```

```
    paid = [r for r in rows if r['status'] == 'paid']
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（6/9）

```
    return {'rows_clean': len(rows), 'paid_rows': len(paid),
```

```
            'paid_sum': round(sum(r['amount'] for r in paid), 2),
```

```
            'cities': len({r['city'] for r in rows})}
```

```
def save_csv(rows, path):
```

```
    with open(path, 'w', encoding='utf-8-sig', newline='') as f:
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（7/9）

```
        w = csv.DictWriter(f, fieldnames=list(rows[0].keys()))
```

w.writeheader()

w.writerows(rows)

```
def save_json(obj, path):
```

```
    with open(path, 'w', encoding='utf-8') as f:
```

```
        json.dump(obj, f, ensure_ascii=False, indent=2)
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（8/9）

```
if __name__ == '__main__':
```

```
    rows = load_orders(DATA / 'orders_dirty.csv')
```

clean, dropped = clean_orders(rows)

s = summarize(clean)

```
    save_csv(clean, DATA / 'orders_clean.csv')
```

```
    save_json({**dropped, **s}, DATA / 'orders_summary.json')
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（9/9）

```
print(dropped, s)
```

验收（做到才算过关，这是 W4 的总验收）：

```
① 脚本一次跑通，data 目录下多出 orders_clean.csv 和 orders_summary.json；
```

② 打印出来的 dropped 和 s 里每个数字，你都能说出它是怎么来的；

```
③ 把 clean_orders 里删掉一行 r['city'] = ...，再跑一遍，说清为什么 cities 变了；
```

```
④ 能讲清 utf-8-sig 和 utf-8 的差别（用 0xef 那个报错讲）。
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

最容易错的地方

★ 往已有文件里补内容要用 'a'；要「读改写」就写新文件再替换，别直接 'w' 覆盖 ——这一步出错，原数据就没了。

★ 裸 except: 会把 KeyboardInterrupt（Ctrl+C）和语法错误一起吞掉，程序该停不停 ——永远写具体类型，至少写 except Exception。

这些句子都是「差一个字、结果差一截」的地方，考前只翻这一页也值。

合上视频，自测一遍（能打勾才算学会）

- [ ] 我能不看资料写出：往已有文件里补内容要用 'a'；要「读改写」就写新文件再替换，别直接 'w' 覆盖 ——这一步出错，原数据就没了。
- [ ] 我能不看资料写出：JSON 不支持 datetime / Decimal / numpy 类型 —— 直接 dump 会报
- [ ] 我能不看题源，把今天用到的每个函数「为什么要有它」说一遍
- [ ] 我能说出今天每个参数/返回值改了会怎样——换个写法结果差在哪
- [ ] 今天这几段代码我在本机都跑通、并看清了输出

这些句子都是「差一个字、结果差一截」的地方；三个勾以上，今天就算过关。

今天收工前，先知道明天学什么

第 8 天 • numpy：ndarray、向量化、广播、布尔索引

不用提前学。明天点「① 开始今天的学习」，会换上明天那份 PPT。

收工三件事（5 分钟）

- 今天没跑通的脚本存进 90-我的作业\，命名 2026-11-08-第07天-*.py
- 打卡表「③ 每日打卡」记一笔：今天卡在哪、错在哪（一句话就够）
- 错题明天重做一遍，练习数据在 60-练习环境\Python-Env\data

基础期 • 第 8 天 • 11/9

numpy：ndarray、向量化、广播、布尔索引

不用 for 循环，把一份 1200 行的金额列一次性做完「解析 + 判空 + 缩放 + 条件筛选」。

第 8 天 · 11/9 · 今日知识点回顾

numpy：ndarray、向量化、广播、布尔索引

目标：不用 for 循环，把一份 1200 行的金额列一次性做完「解析 + 判空 + 缩放 + 条件筛选」。

对着 B 站视频看：视频讲到的每一句，这张 PPT 里都有一条；复习时只翻这份，不用再回去翻分P。

今天结束时，你应该能做到

验收标准（做到这句就算过关）

- 不用 for 循环，把一份 1200 行的金额列一次性做完「解析 + 判空 + 缩放 + 条件筛选」。

今天的做法（顺序别颠倒）

- 题库地址：file:///D:/DSH工作区/00-学习中心/60-练习环境/Python-Env/data
- 做完记一句：今天卡在哪、错在哪 → 写进 3-每日打卡。

今天看哪几集、读哪一章

这一天的知识：4 集 • 合计约 97 分钟（边看边在本机敲一遍）

P14	numpy的数组的创建	约 16 分钟
P15	数组的计算和数组的计算	约 42 分钟
P17	numpy中的索引和切片	约 18 分钟
P21	numpy中的nan和常用统计方法	约 21 分钟

教材（已下载到 20-教材与讲义\python-data\，断网也能看）

`https://raw.githubusercontent.com/datawhalechina/joyful-pandas/master/source/Content/ch1.rst`

第一章 预备知识：列表推导式、匿名函数、numpy 基础

视频地址就在下面的「今天做哪些题」网页里，点一下直达分P。

知识点 1/7 • ndarray 是「同类型 + 定长」的数组，这是它快的原因

- 一句话：一个 ndarray 里所有元素类型相同，内存连续，所以整列运算能一次性算完。
- 代码：

```
import numpy as np
a = np.array([1, 2, 3, 4])
print(a.dtype, a.shape, a.ndim)      # int64 (4,) 1
b = np.array([[1, 2, 3], [4, 5, 6]])
print(b.shape)                        # (2, 3)  2 行 3 列
print(b.dtype)
```

★ np.array([1, 2, 3.5]) 会整体变成 float64 —— 一个 float 就把全数组升成 float。

- 这和 Python 的 list 完全不同（list 里什么都能混）。

知识点 2/7 • 向量化：不写 for，整列一起算

- 一句话：a + 1、a * 2、a > 3 都是「对每个元素做同一件事」，不用循环。
- 代码：

```
a = np.array([1, 2, 3, 4])
print(a + 1)         # [2 3 4 5]
print(a * 2)         # [2 4 6 8]
print(a > 2)         # [False False  True  True]
print(a[a > 2])      # [3 4]      ← 布尔索引
print(a.sum(), a.mean(), a.max(), a.std())
```

★ 不要写 for i in range(len(a)): b[i] = a[i] * 2 —— 1200 行时差别不明显，到 120 万行时慢 100 倍以上。向量化是 pandas 快过 Excel 的根源。

知识点 3/7 • 广播：形状不同也能算，但有规则

- 一句话：从右往左比形状，要么相等、要么其中一个是 1，否则报错。
- 代码：

```
m = np.array([[1, 2, 3], [4, 5, 6]])     # shape (2,3)
print(m + 10)                             # 标量广播：每个元素 +10
print(m + np.array([10, 20, 30]))         # (3,) 广播到每一行
print(m + np.array([[10], [20]]))         # (2,1) 广播到每一列
# m + np.array([1, 2])                    # ValueError: operands could not be broadcast
```

★ 报错原文：ValueError: operands could not be broadcast together with shapes (2,3) (2,)

- —— 最后一位 3 和 2 既不相等也不是 1，所以失败。看报错先看 shapes。

知识点 4/7 • nan 是「缺失值」，它和任何数比较都是 False

- 一句话：np.nan != np.nan，判空要用 np.isnan() 或 pd.isna()。
- 代码：

```
a = np.array([1.0, np.nan, 3.0])
print(a.sum(), a.mean())          # nan nan   ← 一个 nan 污染整个统计
print(np.nansum(a), np.nanmean(a))   # 4.0 2.0  ← nansum/nanmean 跳过 nan
print(np.isnan(a))                # [False  True False]
print(a == a)                     # [ True False  True]   ← nan 自己都不等于自己
```

★ a == np.nan 永远全 False，a != np.nan 永远全 True —— 这是最经典的错。

- 真实数据里 nan 有三种来源：空字符串、'NULL'、Excel 空单元格，读进来都会变 nan。

知识点 5/7 • 真数据上手：把 amount 列解析成数字数组

- 一句话：str.replace 在 numpy 里也能向量化（np.char 或 pd 的字符串方法），但更快的是 pandas。
- 代码：

```
import numpy as np, pandas as pd, pathlib
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
s = pd.read_csv(DATA / 'orders_dirty.csv')['amount']
num = pd.to_numeric(s.astype(str).str.replace('¥', '', regex=False)
.str.replace(',', '', regex=False), errors='coerce')
print(len(num), num.isna().sum(), round(num.sum(), 2))
```

- （本机实测：1200 行里 35 行金额为空 → 解析后 35 个 nan；
- 全列合计 1,199,463.00。这个数第 12 天 groupby 之后还会再出现一次，如果两次对不上，就是清洗口径变了。）

知识点 6/7 • 形状变换：reshape / 转置 / 拼接

- 一句话：reshape 改形状不改数据；.T 转置；concatenate 按轴拼。
- 代码：

```
a = np.arange(6)
print(a.reshape(2, 3))
print(a.reshape(2, 3).T)                  # 转置
print(np.concatenate([a, a]))             # 拼成一维 12 个
print(np.vstack([a.reshape(2, 3), a.reshape(2, 3)]))   # 竖着摞，变成 4 行
```

★ reshape 的元素个数必须对得上，否则报ValueError: cannot reshape array of size 6 into shape (4,2)。

- 先用 a.size 算一下再 reshape。

知识点 7/7 • numpy 2.x 删了一堆别名，别再抄老教程

- 一句话：np.float_ / np.NaN / np.Inf / np.unicode_ / np.in1d / np.trapz 在本机 2.5.4 上全部 AttributeError。
- 代码：

```
import numpy as np
print(np.__version__)        # 2.5.4
# np.float_          → AttributeError: module 'numpy' has no attribute 'float_'
# np.NaN             → AttributeError
# np.in1d            → AttributeError (改用 np.isin)
# np.trapz           → AttributeError (改用 np.trapezoid)
```

★ 本机实测确认过的替代：np.float64 / np.nan / np.inf / np.str_ / np.isin / np.trapezoid。

- 抄 B 站 2020 年的代码前，先看它用的是哪个版本。

代码速查：今天这几段，要在本机跑通（1/5）

写一个 numpy_probe.py，把下面每一条都跑出真实数字：

```
import numpy as np, pandas as pd, pathlib
```

```
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
```

```
df = pd.read_csv(DATA / 'orders_dirty.csv')
```

1) 向量化：quantity 列整体判断

q = df['quantity'].to_numpy()

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（2/5）

```
print('数量为负的行数：', (q < 0).sum())          # 应为 25
```

```
print('数量分布：', np.unique(q, return_counts=True))
```

2) 广播：给每个金额打九折

```
num = pd.to_numeric(df['amount'].astype(str).str.replace('¥', '', regex=False)
```

```
                    .str.replace(',', '', regex=False), errors='coerce')
```

arr = num.to_numpy()

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（3/5）

```
print('九折后合计：', round(np.nansum(arr) * 0.9, 2))
```

3) nan 处理：三种统计口径对比

```
print('含 nan 直接 sum：', arr.sum())
```

```
print('nansum：', round(np.nansum(arr), 2))
```

```
print('nan 个数：', np.isnan(arr).sum())          # 应为 35
```

4) 布尔索引：筛出金额 > 2000 的行

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（4/5）

big = df[num > 2000]

```
print('金额 > 2000 的行数: ', len(big))
```

```
print(big[['order_id', 'amount']].head(3).to_string(index=False))
```

验收（做到才算过关）：

① 数量为负的行数 = 25，nan 个数 = 35，两个数都要对上；

```
② 能说清 arr.sum() 为什么是 nan，而 np.nansum 能给出数；
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（5/5）

```
③ 把 (q < 0).sum() 改成 sum(q < 0)，看结果一样吗，并解释为什么。
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

最容易错的地方

★ 不要写 for i in range(len(a)): b[i] = a[i] * 2 —— 1200 行时差别不明显，到 120 万行时慢 100 倍以上。向量化是 pandas 快过 Excel 的根源。

★ 报错原文：ValueError: operands could not be broadcast together with shapes (2,3) (2,)

—— 最后一位 3 和 2 既不相等也不是 1，所以失败。看报错先看 shapes。

★ a == np.nan 永远全 False，a != np.nan 永远全 True —— 这是最经典的错。

这些句子都是「差一个字、结果差一截」的地方，考前只翻这一页也值。

合上视频，自测一遍（能打勾才算学会）

- [ ] 我能不看资料写出：np.array([1, 2, 3.5]) 会整体变成 float64 —— 一个 float 就把全数组升成 float。
- [ ] 我能不看资料写出：a == np.nan 永远全 False，a != np.nan 永远全 True —— 这是最经典的错。
- [ ] 我能不看题源，把今天用到的每个函数「为什么要有它」说一遍
- [ ] 我能说出今天每个参数/返回值改了会怎样——换个写法结果差在哪
- [ ] 今天这几段代码我在本机都跑通、并看清了输出

这些句子都是「差一个字、结果差一截」的地方；三个勾以上，今天就算过关。

今天收工前，先知道明天学什么

第 9 天 • pandas 读数据 + Series/DataFrame + 一眼看清一张表

不用提前学。明天点「① 开始今天的学习」，会换上明天那份 PPT。

收工三件事（5 分钟）

- 今天没跑通的脚本存进 90-我的作业\，命名 2026-11-09-第08天-*.py
- 打卡表「③ 每日打卡」记一笔：今天卡在哪、错在哪（一句话就够）
- 错题明天重做一遍，练习数据在 60-练习环境\Python-Env\data

基础期 • 第 9 天 • 11/10

pandas 读数据 + SeriesDataFrame + 一眼看清一张表

拿到任意一份 CSV，30 秒内说出：几行几列、每列什么类型、缺多少、数值列的范围。

第 9 天 · 11/10 · 今日知识点回顾

pandas 读数据 + Series/DataFrame + 一眼看清一张表

目标：拿到任意一份 CSV，30 秒内说出：几行几列、每列什么类型、缺多少、数值列的范围。

对着 B 站视频看：视频讲到的每一句，这张 PPT 里都有一条；复习时只翻这份，不用再回去翻分P。

今天结束时，你应该能做到

验收标准（做到这句就算过关）

- 拿到任意一份 CSV，30 秒内说出：几行几列、每列什么类型、缺多少、数值列的范围。

今天的做法（顺序别颠倒）

- 题库地址：file:///D:/DSH工作区/00-学习中心/60-练习环境/Python-Env/data
- 做完记一句：今天卡在哪、错在哪 → 写进 3-每日打卡。

今天看哪几集、读哪一章

这一天的知识：4 集 • 合计约 42 分钟（边看边在本机敲一遍）

P15	Pandas2数据读取	约 14 分钟
P16	Pandas2读取CSV	约 16 分钟
P20	Pandas2数据信息查看操作	约 6 分钟
P21	Pandas2统计计算操作	约 6 分钟

教材（已下载到 20-教材与讲义\python-data\，断网也能看）

`https://raw.githubusercontent.com/datawhalechina/joyful-pandas/master/source/Content/ch2.rst`

第二章：文件的读取与写入、Series/DataFrame、常用基本函数

视频地址就在下面的「今天做哪些题」网页里，点一下直达分P。

知识点 1/7 • read_csv 的四个参数决定成败

- 一句话：encoding、dtype、parse_dates、na_values —— 四个参数一次给全，后面少踩一半坑。
- 代码：

```
import pandas as pd, pathlib
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
df = pd.read_csv(DATA / 'orders_dirty.csv', encoding='utf-8-sig')
print(df.shape)          # (1200, 10)
print(list(df.columns))
print(df.dtypes)
```

★ 不加 encoding='utf-8-sig' 时，第一列列名会变成 '\ufefforder_id'，

- 后面 df['order_id'] 直接 KeyError: 'order_id' —— 报错信息里那个方框字符就是 BOM。

这一页只讲「read_csv 的四个参数决定成败」。讲完就翻页，下一页是另一个知识点。

知识点 2/7 • head / info / describe / value_counts：四眼看全表

- 一句话：head 看长相、info 看类型和缺失、describe 看数值分布、value_counts 看分类计数。
- 代码：

```
print(df.head(3).to_string())
print(df.info())
print(df[['quantity']].describe().round(2))
print(df['status'].value_counts(dropna=False))
```

★ info() 输出的「Non-Null Count」比总行数少多少，就是这一列缺多少。

- 比 isnull().sum() 更省一次调用，且一眼看到 dtype —— 脏数据最常见的错就是「金额列被读成了 object」。

知识点 3/7 • Series 和 DataFrame 的关系

- 一句话：DataFrame 是一张表；df['col'] 取出的是 Series（一列，带索引）。
- 代码：

```
s = df['city']
print(type(s))                    # <class 'pandas.Series'>
print(s.shape, s.dtype)           # (1200,) object
print(s.head(3).tolist())
print(df[['city']].shape)         # (1200, 1) ← 双层方括号拿到的是 DataFrame
```

★ df['city'] 是 Series，df[['city']] 是 DataFrame。两者长得像，但

- df['city'].mean() 能用、df[['city']].mean() 返回 Series —— 差一层方括号，行为不同。

知识点 4/7 • 索引：默认 0..n-1，可以换

- 一句话：df.index 是行号，set_index 能把它换成业务主键。
- 代码：

```
print(df.index[:3])                      # RangeIndex(start=0, stop=3, step=1)
d2 = df.set_index('order_id')
print(d2.index[:3].tolist())
print(d2.loc[d2.index[0], ['city', 'amount']].to_dict())
```

★ set_index 之后 order_id 不再是普通列。想保留成列要用 reset_index() 还原，或者用 df.set_index('order_id', drop=False)。

知识点 5/7 • 真数据的「第一印象」：本机 1200 行订单

- 一句话：先跑一遍四眼，把所有异常记下来，别急着清洗。
- 代码：

```
print('行数', len(df))
print('重复 order_id', df['order_id'].duplicated().sum())      # 37
print('金额缺失', df['amount'].isna().sum())                   # 35
print('城市缺失', df['city'].isna().sum())                     # 56
print('状态取值', df['status'].unique())                       # 6 种写法
print('日期前缀分布', df['create_time'].astype(str).str[:4].value_counts().to_dict())
```

- （本机实测这 6 个数：1200 行 / 37 个重复单号 / 35 行金额空 / 56 行城市空 /status 有 PAID、Paid、paid、'paid '、pending、refunded 六种写法。
- 第 10 天就是专门洗这六个问题。）

知识点 6/7 • 数值列的三种统计口径要分清

- 一句话：mean 跳过 nan，sum 也跳过 nan（pandas 默认 skipna=True），和 numpy 相反。
- 代码：

```
import numpy as np
s = pd.Series([1.0, np.nan, 3.0])
print(s.sum(), s.mean(), s.count(), len(s))    # 4.0 2.0 2 3
print(s.sum(skipna=False))                      # nan  ← 关掉跳过
```

★ 这条和 numpy 的 arr.sum() 行为相反**：numpy 遇 nan 就变 nan，pandas 默认跳过。**

- 混用两套 API 时，同一个「合计」会算出两个数。

知识点 7/7 • 读 Excel 只多一个参数

- 一句话：pd.read_excel(path, sheet_name=...)；不给 sheet_name 读第一个 sheet。
- 代码：

```
xl = pd.ExcelFile(DATA / 'sales_dirty.xlsx')
print(xl.sheet_names)                    # ['明细', '商品表', '城市表']
d = pd.read_excel(DATA / 'sales_dirty.xlsx', sheet_name='明细')
print(d.shape, list(d.columns))
```

★ 本机实测：明细 sheet 400 行 9 列，列名是「订单号/日期/城市/品类/商品/数量/单价/金额/销售员」。

- 读 Excel 比 CSV 慢 5–10 倍，能导出 CSV 就先导。

代码速查：今天这几段，要在本机跑通（1/3）

写一个 look_at_data.py，对三份数据各跑一遍「四眼」：

```
import pandas as pd, pathlib
```

```
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
```

```
def look(path, **kw):
```

```
    df = pd.read_csv(path, encoding='utf-8-sig', **kw)
```

print('=' * 60)

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（2/3）

```
    print(path.name, df.shape)
    print(df.dtypes.to_string())
    print('缺失: ', df.isna().sum()[df.isna().sum() > 0].to_dict())
    num = df.select_dtypes('number')
    if not num.empty:
        print(num.describe().round(2).to_string())
    return df
o = look(DATA / 'orders_dirty.csv')
u = look(DATA / 'users_dirty.csv')
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（3/3）

e = look(DATA / 'events_dirty.csv')

验收（做到才算过关）：

① orders 报出 1200 行 10 列、37 个重复单号、35 行金额缺失、56 行城市缺失；

② users 报出 500 行 7 列，并说出 age 列的最大值是多少（提示：会有 999）；

③ 能说出为什么 amount 列被读成了 object 而不是 float64。

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

最容易错的地方

后面 df['order_id'] 直接 KeyError: 'order_id' —— 报错信息里那个方框字符就是 BOM。

比 isnull().sum() 更省一次调用，且一眼看到 dtype —— 脏数据最常见的错就是「金额列被读成了 object」。

这些句子都是「差一个字、结果差一截」的地方，考前只翻这一页也值。

合上视频，自测一遍（能打勾才算学会）

- [ ] 我能不看资料写出：info() 输出的「Non-Null Count」比总行数少多少，就是这一列缺多少。
- [ ] 我能不看资料写出：df['city'] 是 Series，df[['city']] 是 DataFrame。两者长得像，但
- [ ] 我能不看资料说出：一句话：encoding、dtype、parse_dates、na_values —— 四个参数一次给全，后面少踩一半坑。
- [ ] 我能不看资料说出：一句话：先跑一遍四眼，把所有异常记下来，别急着清洗。
- [ ] 我能不看题源，把今天用到的每个函数「为什么要有它」说一遍
- [ ] 我能说出今天每个参数/返回值改了会怎样——换个写法结果差在哪

这些句子都是「差一个字、结果差一截」的地方；三个勾以上，今天就算过关。

今天收工前，先知道明天学什么

第 10 天 • 清洗第一课：缺失值、重复行、类型转换、字符串规整

不用提前学。明天点「① 开始今天的学习」，会换上明天那份 PPT。

收工三件事（5 分钟）

- 今天没跑通的脚本存进 90-我的作业\，命名 2026-11-10-第09天-*.py
- 打卡表「③ 每日打卡」记一笔：今天卡在哪、错在哪（一句话就够）
- 错题明天重做一遍，练习数据在 60-练习环境\Python-Env\data